

Factorized Embeddings & Weight Sharing.

ALBERT cuts BERT-base from $\sim 110\text{M}$ parameters to $\sim 12\text{M}$ with two ideas: factor the $V \times H$ embedding through a tiny bottleneck E , and make all L encoder layers share one weight tensor. We derive both savings and total them line by line — every count multiplied out, nothing rounded away.

THE EQUATIONS

$$\underbrace{V \cdot H \longrightarrow V \cdot E + E \cdot H}_{\text{factorized embedding}} \qquad \underbrace{L \cdot P_\ell \longrightarrow P_\ell}_{\text{cross-layer sharing}}$$

Worked example. $V=30000$, $H=768$, $E=128$, $L=12$; per-layer params $\approx 7.09\text{M}$.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: same architecture, far fewer parameters	2
2	Factorized embedding parameterization	3
3	Cross-layer parameter sharing	3
4	Combine: ALBERT-base vs BERT-base, line by line	4
5	Properties that matter	4
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The claim: same architecture, far fewer parameters

ALBERT (*A Lite BERT*) keeps BERT's Transformer encoder but attacks the two largest parameter sinks. The token embedding is enormous because V is large; ALBERT factors it through a bottleneck $E \ll H$. The encoder repeats L near-identical layers; ALBERT makes them *literally identical* by tying their weights.

$$\text{params}_{\text{emb}} = V \cdot E + E \cdot H \quad \text{params}_{\text{enc}} = P_\ell \text{ (one shared layer, not } L \cdot P_\ell) \quad (1)$$

Symbol	Value	Meaning
V	30000	vocabulary size
H	768	hidden / model width
E	128	embedding bottleneck dimension ($E \ll H$)
L	12	number of encoder layers
P_ℓ	$\approx 7.09\text{M}$	parameters in one encoder layer

WordPiece embeddings are *context-free* — they only need to encode the identity of a token. Forcing them to live in the full $H=768$ space wastes capacity. The contextual work happens in the attention layers. So shrink the lookup table to a thin $E=128$ space, then let one small matrix project up to H when the token enters the encoder.

2 Factorized embedding parameterization

A standard BERT embedding is a single $V \times H$ matrix: one H -dimensional vector per vocabulary entry. ALBERT splits it into a $V \times E$ lookup followed by an $E \times H$ projection shared by all tokens.

2.1 Worked: $V \cdot H$ vs $V \cdot E + E \cdot H$

Standard:

$$V \cdot H = 30000 \times 768 = 23,040,000.$$

Factorized:

$$V \cdot E + E \cdot H = \underbrace{30000 \times 128}_{3,840,000} + \underbrace{128 \times 768}_{98,304} = 3,938,304.$$

$$\text{ratio} = \frac{23,040,000}{3,938,304} = 5.85 \times \text{fewer parameters,} \quad \text{saved } 19,101,696.$$

The projection $E \cdot H$ is negligible ($\sim 98\text{k}$); essentially all the saving comes from the lookup table dropping from width 768 to width 128.

Factorization decouples the vocabulary size from the hidden size. You can now grow H for more modelling capacity *without* paying $V \cdot \Delta H$ extra embedding parameters — the embedding table only ever scales with E .

3 Cross-layer parameter sharing

BERT stores L independent encoder layers. ALBERT stores *one* and applies it L times in sequence (all-shared variant). The encoder parameter count divides by L .

3.1 Anatomy of one layer ($\approx 7.09\text{M}$)

With $H = 768$, one Transformer encoder layer holds:

Sub-block	Formula	Params
Attention Q, K, V, O	$4(H^2 + H) = 4(768^2 + 768)$	2,362,368
FFN up $H \rightarrow 4H$	$H \cdot 4H + 4H = 768 \cdot 3072 + 3072$	2,362,368
FFN down $4H \rightarrow H$	$4H \cdot H + H = 3072 \cdot 768 + 768$	2,360,064
2 LayerNorms	$2 \times 2H = 4 \times 768$	3,072
Per layer P_ℓ		7,087,872

3.2 The sharing saving

$$\text{BERT encoder} = L \cdot P_\ell = 12 \times 7,087,872 = 85,054,464 (\approx 85.05\text{M}).$$

$$\text{ALBERT encoder} = P_\ell = 7,087,872 (\approx 7.09\text{M}) \Rightarrow 12 \times \text{fewer.}$$

3.3 Heatmap: per-block share of one layer

	Attn	FFN up	FFN down	LayerNorm
fraction of P_ℓ	0.333	0.333	0.333	0.0004

(The three dense projections dominate; LayerNorm scale/bias is negligible.)

4 Combine: ALBERT-base vs BERT-base, line by line

Adding positional (512×768), segment (2×768) and embedding-LayerNorm (2×768) terms to the token embedding, then the encoder:

Component	BERT-base	ALBERT-base
Token embedding	$30000 \cdot 768 = 23,040,000$	$30000 \cdot 128 = 3,840,000$
Embedding projection $E \rightarrow H$	—	$128 \cdot 768 = 98,304$
Position embedding	$512 \cdot 768 = 393,216$	393,216
Segment embedding	$2 \cdot 768 = 1,536$	1,536
Embedding LayerNorm	$2 \cdot 768 = 1,536$	1,536
Embedding subtotal	23,436,288	4,334,592
Encoder ($L = 12$ layers)	$12 \cdot 7,087,872 = 85,054,464$	$1 \cdot 7,087,872 = 7,087,872$
Total	108,490,752	11,422,464
	$\approx 108.5\text{M}$ ($\approx 110\text{M}$)	$\approx 11.4\text{M}$ ($\approx 12\text{M}$)

The overall reduction is $108,490,752 / 11,422,464 = 9.5\times$. Of the $\sim 97\text{M}$ saved, $\sim 78\text{M}$ comes from layer sharing and $\sim 19\text{M}$ from the factorized embedding.

Fewer parameters does **not** mean less compute. Sharing weights does not change the forward pass — the encoder still executes $L = 12$ layers, so the FLOPs (and inference latency) of ALBERT-base match BERT-base almost exactly. ALBERT saves *memory* and *download size*, not wall-clock time per token. If you need speed, that is distillation's job (Topic 1), not parameter sharing.

5 Properties that matter

1. **Orthogonal savings.** Factorization shrinks the embedding ($\div 5.85$); sharing shrinks the encoder ($\div L$). They compose to $\sim 9.5\times$ overall.
2. **Capacity vs. size decoupled.** With sharing, you can widen H or deepen L for more capacity at near-constant parameter cost — ALBERT-xxlarge has $H=4096$ yet only $\sim 235\text{M}$ params.
3. **Regularisation effect.** Tying layers constrains the function class, which can improve generalisation and training stability.

4. **Same compute.** FLOPs are unchanged — a deliberate trade of memory for nothing on the latency axis.

6 Where this sits in the track

Connects to	How
Module 1.1 (BERT)	the 110M baseline ALBERT compresses
Topic 0.2.3 (FFN param count)	the per-layer 7.09M is dominated by the $4H$ FFN derived there
Module 1.3 Topic 1	distillation = smaller student; sharing = reused weights — two distinct compression axes (size vs. speed)

A Reproducible (NumPy)

Inputs: $V=30000$, $H=768$, $E=128$, $L=12$, $P=512$ positions, 2 segments.

```
V, H, E, L, P, S = 30000, 768, 128, 12, 512, 2
std = V * H                                # 23,040,000
fac = V * E + E * H                        # 3,938,304
print(std, fac, round(std / fac, 2))        # ... 5.85x

per_layer = 4*(H*H + H) + (H*4*H + 4*H) + (4*H*H + H) + 2*(2*H)
print(per_layer)                           # 7,087,872

bert_emb = V*H + P*H + S*H + 2*H          # 23,436,288
alb_emb  = V*E + E*H + P*H + S*H + 2*H    # 4,334,592
bert = bert_emb + L*per_layer              # 108,490,752 (~110M)
alb  = alb_emb + per_layer                 # 11,422,464 (~12M)
print(bert, alb, round(bert / alb, 1))      # ... 9.5x
```